Daniel Brenner, Dinh Xuan Truong Ung, Amine Rekik, Roman Pauli

Analyseaufgabe 01 – Gottklassen erkennen

Annahmen und Voraussetzungen:

Pfad zur File

Der Pfad, welcher in der val-Variable „jarFile“ (Zeile 58) angegeben ist, sollte auf eine gültige Datei zeigen. Hierbei stellen wir zwei Dateien bereit, wovon eine die aus dem Beispiel ist (pdfbox) und die andere eine eigene (Test), deren Inhalt in dem Bereich Quellcode_Test angeschaut werden kann.

Fehlermeldungen: Es wurde keine File gefunden => IllegalStateException() wird geworfen.

Es wurde eine ungültige File gefunden => IllegalStateException() wird geworfen

Input für Limits

Es gibt die Möglichkeit entweder Default-Limits (Eingabe von 1) zu verwenden oder eigene einzutragen (Eingabe von 0), nachdem das Programm gestartet wurde. Die Default-Limits können im Quellcode verändert werden (Z. 20-25). Nur ermittelte Werte, die größer als die Limits sind, werden bei der Detektion von Gottklassen berücksichtigt (bspw. setMethodLimit = 1; Klassen, die nur eine Methode haben (einschließlich Konstruktor!) sind nicht zu „groß“). Die Variable setMaxPropertiesFalse ermöglicht die Steuerung, wie viele Limits überzogen werden dürfen, bevor eine Klasse als Gottklasse ausgezeichnet wird (Auch hier gilt: Anzahl muss größer als der gesetzte sein).

Default-Limits:

- Werte = 0 => Limits werden nicht überprüft.
- Werte > 0 => Limits werden gesetzt.
- Werte < 0 => Limit ist ungültig gesetzt und das Programm sollte entsprechend eine Fehlermeldung auswerfen.

Eigene-Limits:

- Werte >= 0 => wie oben
- Werte < 0 => Es wird so lange gefragt, bis ein gültiger Wert eingetragen wird.

setMaxPropertiesFalse:

- Werte = 0 => Wenn ein Limit überzogen wird, ist die Klasse eine Gottklasse
- Werte < 0 oder Werte >= tatsächlich gesetzte Limits (>0) => ungültig
- Werte > 0 => Wenn „Wert“ + 1 an Limits überzogen wird -> Gottklasse

Sonderfall Kohesion: Wert muss zwischen 0 und 1 liegen.

Fehlermeldungen: Limit ungültig gesetzt => IllegalStateException

setMaxPropertiesFalse ungültig gesetzt => IllegalStateException

Konkrete Nutzung:

Zu Beginn erhält man eine kleine Willkommensnachricht und anschließend eine Abfrage, ob man eigene Limits setzten möchte oder die im Quellcode gesetzten Limits nutzten will. Falls man sich für eigene Werte entscheidet, muss man erst für die fünf Limits (Methoden_Anzahl, Felder_Anzahl, Instruktionen_Anzahl, Kohesion, Kopplung) nacheinander Werte eingeben und danach die Grenze setzen, ab wie vielen überzogenen Limits eine Klasse eine Gottklasse sein soll. Für alle Limits sind dabei Werte von = 0 gleichbedeutend mit „nicht gesetzt“ und Werte von >0 gesetzt. Für den Sonderfall, Kohesion, gilt, dass der Wert im Intervall [0,1] liegen muss. Für die Grenze zur Detektion einer Gottklasse „setMaxPropertiesFalse“ wird auch berücksichtigt, dass die Grenze nicht höher gesetzt werden kann, als die Anzahl der gesetzten Limits. Normalerweise sollten falsch gesetzte Limits vom Programm erkannt werden und den Nutzer darauf aufmerksam machen.
Danach berechnet das Programm für jede Klasse, ob die Limits erreicht werden. Zum Schluss werden die erkannten Gottklassen mit ihren jeweiligen Eigenschaften nochmal ausgegeben.

Konfigurationsmöglichkeiten:

Man kann die Dateien austauschen und den jeweiligen Pfad zu dieser Datei in der Variable jarFile austauschen (Z. 59).

Die Default-Limits für die einzelnen Eigenschaften können beliebig geändert werden im Quellcode (Z. 20-25).

Falls man möchte, dass ab dem erreichen des gesetzten Limits bereits der Wert als zu groß angesehen werden soll, könnte man in der Hilfsmethode (isWrongful (ca. Zeile 237)) in der jeweiligen IF-clause check.toInt < value zu check.toInt <= value ändern (für die Kohesion in der anderen isWrongful Methode weiter unten).